DEPI Project

Real-Time Anomaly Detection & Alerting System (IoT / Time-Series)

Group Members

1. **Menna Reda Fadl** **(Team Leader)**
2. Eman Amr Galala
3. Reem Mostafa Mohamed Kamal
4. Renada Ahmed Anwar
5. Manar Ali Hammad
6. Mariam Ahmed Mohamed

1.Project Planning &Management

Project Overview

The IoT Real-Time Anomaly Detection System is a smart system designed to monitor IoT sensor data and detect abnormal behavior in real time using machine learning techniques. The system collects live data from sensors, analyzes it continuously, and identifies unusual activities such as device failures or suspicious behavior. When an anomaly is detected, the system sends instant alerts to improve security, reliability, and performance in IoT environments.

Objectives

1. Detect anomalies in real-time
2. Reduce downtime
3. Provide automatic alerts
4. Visualize sensor behavior
5. Build scalable ML pipeline

Scope

A. In Scope

1. Data ingestion from simulated or real loT sensors
2. Streaming processing pipeline
3. Machine learning anomaly detection model
4. Alerting system
5. Monitoring dashboard

B. Out of Scope

1. Hardware manufacturing of sensors
2. Mobile application development

Timeline

Phase	Duration	Description
Research & Design	Week 1	Architecture & dataset
Data Pipeline	Week 2-3	Streaming ingestion
Processing	Week 4	Cleaning & features
Model Development	Week 5-6	Train model
Real-Time Inference	Week 7	Deploy streaming
Alerting	Week 8	Notifications
Dashboard	Week 9	Visualization
Testing	Week 10	Evaluation

Roles

Role	Responsibility	Member
Data Engineer	Data Collection & Processing Pipeline	Renada Ahmed
Machine Learning Engineer	Model Development & Training	Manar Ali
Backend Engineer	API Development & Alert System	Eman Amr
Cloud Engineer	Cloud Infrastructure & Resource Management	Menna Reda
MLOps Engineer	Model Deployment & Monitoring	Reem Mostafa
Visualization & Product Engineer	Dashboards & Data Insights	Mariam Ahmed

Risk Assessment

Risk	Mitigation
No dataset	Simulated/public data
False positives	Threshold tuning
Latency	Streaming frameworks
Data drift	Retraining
Failure	Monitoring

KPIs

Metric	Target
Accuracy	>92%
False Positive Rate	<5%
Alert Delay	<5 sec
Uptime	99%
Throughput	>1000 events/sec

Milestone

Milestone 1: Data Collection, Preprocessing, and Exploration

Objectives

- Collect real or simulated time-series IoT data
- Preprocess and explore seasonal patterns and noise

Tasks

1. Data Collection: a. Ingest data from multiple sensors (temperature, pressure, etc.) via simulated streams or public IoT datasets. b. Store raw data in Azure Data Lake or Blob Storage.

2. Data Preprocessing: a. Handle missing values, smoothing, and window segmentation. b. Create engineered features (lag features, rolling means).

3. Exploratory Data Analysis (EDA): a. Analyze trends, seasonal effects, and noise patterns. b. Identify baseline normal behavior and potential anomalies.

Deliverables

- Cleaned and Structured Time-Series Dataset
- Data Preprocessing Pipeline Documentation
- EDA Report with Visualizations

Milestone 2: Model Development & Evaluation Objectives

- Train reliable anomaly detection models suitable for streaming data

Tasks

1. Model Development: a. Implement models such as LSTM Autoencoders, Isolation Forests, or Prophet-based threshold detection.

2. Evaluation: a. Evaluate using precision, recall, F1-score for anomaly classes. b. Test on held-out test sets and synthetic anomalies.

3. Optimization: a. Adjust thresholds and retrain pipelines for robustness.

Deliverables

- Trained Anomaly Detection Models
- Model Evaluation Report

Milestone 3: Advanced Techniques & Cloud Integration Objectives

- Deploy the model in real time on Azure

Tasks

1. Cloud Deployment: a. Use Azure Stream Analytics and Azure Functions for real-time scoring. b. Containerize model inference with Docker deployed through Azure Container Instances.

2. REST API: a. Build a REST API for real-time anomaly scoring.

3. Integration: a. Integrate with Azure Event Hubs for stream ingestion.

Deliverables

- Deployed Real-Time Anomaly Detection Service
- Real-Time API Endpoints

Milestone 4: MLOps, Monitoring, & Dashboard Objectives

- Implement production-grade monitoring, alerts, and retraining automation

Tasks

1. MLOps Tracking: a. Track model versions and experiments with MLflow or Azure ML.

2. Monitoring: a. Use Azure Monitor and custom metrics for drift and performance monitoring.

3. Dashboard: a. Build a dashboard (Power BI or Grafana) showing anomaly rates in real time.

4. Retraining Plan: a. Set up scheduled retraining pipelines triggered by drift detection.

Deliverables

- Monitoring & Alerting Setup
- Dashboard & KPI Visualization
- MLOps Pipeline Documentation

Milestone 5: Final Documentation & Presentation Objectives

- Summarize and demonstrate end-to-end solution

Tasks

1. Final Report: a. Document pipeline, deployment, challenges, and business impact.

2. Presentation: a. Live demo of streaming anomaly detection with alerts and dashboards.

3. Future Enhancements: a. Propose edge deployment optimization or predictive maintenance features.

Deliverables

- Final Project Report
- Final Presentation Deck
- Future Improvements Document

2. Literature Review

Introduction to IoT and Anomaly Detection

The rapid growth of the Internet of Things (IoT) has led to the deployment of millions of smart devices in areas such as healthcare, smart cities, industrial automation, and environmental monitoring. These devices continuously generate large volumes of real-time sensor data, making IoT systems highly vulnerable to abnormal behavior, faults, and cyberattacks. As a result, real-time anomaly detection has become an essential research area for ensuring system reliability, security, and efficiency.

Traditional and Machine Learning Approaches

Traditional anomaly detection techniques relied on statistical methods and rule-based systems. These approaches were effective for small datasets but faced limitations when handling high-dimensional and continuously streaming IoT data. Researchers later introduced Machine Learning (ML) techniques such as Decision Trees, Support Vector Machines (SVM), and K-Nearest Neighbors (KNN) to improve anomaly detection accuracy. However, these models often require labeled datasets and may struggle with unseen anomalies.

Deep Learning Techniques for IoT Anomaly Detection

Recently, Deep Learning approaches have shown significant improvements in IoT anomaly detection systems. Autoencoders, Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Generative Adversarial Networks (GANs) are widely used because of their ability to learn complex patterns from sensor data automatically. Among these techniques, Autoencoders are particularly popular for anomaly detection because they can reconstruct normal data patterns and identify abnormal data based on reconstruction error.

Real-Time IoT Monitoring Systems and Technologies

Several studies have implemented real-time anomaly detection systems using IoT sensors connected to cloud platforms.These systems commonly use MQTT or HTTP protocols for communication and frameworks such as FastAPI, TensorFlow, or PyTorch for backend processing. Researchers also emphasize the importance of low-latency processing and scalability when handling streaming sensor data.

Challenges and Research Motivation

Despite the progress in this field, many existing systems still face challenges including high computational cost, false positive alerts, limited scalability, and difficulty detecting unknown anomalies in real-time environments. Therefore, this project aims to develop an efficient IoT Real-Time Anomaly Detection system capable of monitoring sensor data streams, detecting abnormal behavior instantly, and providing real-time alerts with high accuracy and low response time.

3. Requirements Gathering

1. Stakeholder Analysis

The following stakeholders have been identified for this Academic Project Management & Documentation System. Each stakeholder has specific needs, influence levels, and expectations that directly impact system requirements.

Stakeholder Table

Stakeholder	Company / Organization	Role / Title	Key Needs & Expectations	Influence Level	Communication Channel
Michael Chen	TechFlow Solutions Inc.	Chief Technology Officer (CTO)	- Ensure system aligns with company tech strategy - Reduce documentation time by 30% - Approve final architecture and deployment	Very High	Weekly Exec Meeting, Slack, Email

Stakeholder	Company / Organization	Role / Title	Key Needs & Expectations	Influence Level	Communication Channel
Linda Rodriguez	TechFlow Solutions Inc.	Product Manager	- Define feature roadmap - Prioritize user stories - Ensure KPIs (uptime, response time, adoption) are met	Very High	Jira, Confluence, Daily Standup
James Okafor	TechFlow Solutions Inc.	Lead Software Engineer	- Oversee code quality and architecture (MVC/Microservices) - Manage GitHub branching strategy (GitFlow) - Ensure CI/CD pipeline works	High	GitHub, Jenkins, Slack #dev-channel
Sarah Vancou	TechFlow Solutions Inc.	Senior QA Engineer	- Define test cases and test plans - Set up automated testing framework - Track bugs and regression results	High	Jira, TestRail, Weekly Bug Review

Stakeholder	Company / Organization	Role / Title	Key Needs & Expectations	Influence Level	Communication Channel
David Kim	TechFlow Solutions Inc.	UX/UI Designer	- Create wireframes and mockups - Define UI/UX guidelines (colors, typography, accessibility) - Conduct usability testing	Medium	Figma, Miro, UserTesting.com

Stakeholder Categories

To make it even clearer for your documentation, you can group stakeholders by category:

Category	Stakeholders Included
Internal Executives	CTO, Product Manager
Internal Development Team	Lead Engineer, QA Engineer, UX Designer
External Partners	Nexus IT Outsourcing (contractor), CloudScale Hosting (infrastructure)
End Users / Clients	Global Finance Corp. (client representative)
Compliance & Security	SecureData Consulting (auditor)

Stakeholder Power-Interest Grid

Interest \ Power	Low Power	High Power
High Interest	UX Designer, QA Engineer	CTO, Product Manager, Lead Engineer, Key Client
Low Interest	External Contractor, Cloud Hosting Partner	Security Auditor

High Power + High Interest = Manage Closely (CTO, Product Manager, Lead Engineer, Key Client)
High Power + Low Interest = Keep Satisfied (Security Auditor)
Low Power + High Interest = Keep Informed (UX, QA)
Low Power + Low Interest = Monitor (Contractor, Hosting Partner)

Sample Stakeholder Needs in Business Language

"I need to see real-time KPIs like system uptime and user adoption rate without asking the dev team every day."
— Linda Rodriguez, Product Manager

"The GitHub repo must follow GitFlow with proper commit messages. No exceptions."
— James Okafor, Lead Software Engineer

"If the system isn't ISO 27001 compliant, our client Global Finance won't sign off."
— Michael Chen, CTO

"I want a simple user manual – my team doesn't read long technical documents."
— Priya Mehta, Key Client (Global Finance Corp.)

Stakeholder Needs Summary

- Lecturer: Needs a clear, structured, and gradable submission format with easy access to all sections.
- Students: Need guidance, templates, and automation to reduce documentation overhead.
- TA & HOD: Need visibility into project progress without micromanaging.
- Examiner: Needs a professional, complete, and exportable final document.

2. User Stories

All user stories follow the standard format:
"As a [type of user], I want [some goal] so that [some reason]."

Priority Legend

- High (H) – Must-have for system to be useful
- Medium (M) – Important but can be delivered later
- Low (L) – Nice to have

User Stories for Lecturer / Evaluator

ID	User Story	Priority
US-LEC-01	As a lecturer, I want to view all student projects in one dashboard so that I can quickly access submissions without searching through emails.	H
US-LEC-02	As a lecturer, I want to leave inline comments on any section (e.g., SRS, diagrams, code) so that students receive specific feedback directly on their work.	H
US-LEC-03	As a lecturer, I want to assign a grade to each section and see a calculated final grade so that grading is transparent and consistent.	H
US-LEC-04	As a lecturer, I want to export the entire project as a single PDF or Word document so that I can archive or share it with external examiners.	M
US-LEC-05	As a lecturer, I want to see a change log or version history so that I can verify students made required improvements.	M

User Stories for Student / Project Manager

ID	User Story	Priority
US-PM-01	As a project manager, I want to create a project timeline using a Gantt chart so that I can visualize milestones and deliverables.	H
US-PM-02	As a project manager, I want to assign roles and tasks to team members so that everyone knows their responsibilities.	H
US-PM-03	As a project manager, I want to define KPIs (e.g., response time, uptime, adoption rate) so that I can measure project success objectively.	H
US-PM-04	As a project manager, I want to log risks and mitigation plans so that the team is prepared for potential issues.	H
US-PM-05	As a project manager, I want to generate a progress report automatically so that I can present status updates to the lecturer.	M

User Stories for Student / Developer

ID	User Story	Priority
US-DEV-01	As a developer, I want to link my GitHub repository so that my source code is version-controlled and accessible.	H
US-DEV-02	As a developer, I want to upload or draw UML diagrams (use case, class, sequence) so that I can complete the design section without external tools.	H
US-DEV-03	As a developer, I want to write and save technical documentation (API, schema, architecture) so that my system is properly documented for future maintenance.	H
US-DEV-04	As a developer, I want to see coding standards and error handling guidelines so that my code follows best practices.	M
US-DEV-05	As a developer, I want to commit code directly from the system to GitHub so that I don't need to switch between platforms.	L

User Stories for Student / Tester

ID	User Story	Priority
US-TST-01	As a tester, I want to create test cases with expected and actual results so that I can systematically verify system functionality.	H
US-TST-02	As a tester, I want to log bugs with severity levels (critical, major, minor) so that developers prioritize fixes correctly.	H
US-TST-03	As a tester, I want to link each bug to a specific requirement or test case so that traceability is maintained.	M
US-TST-04	As a tester, I want to run automated test scripts (if applicable) so that regression testing is faster.	L

User Stories for Teaching Assistant

ID	User Story	Priority
US-TA-01	As a teaching assistant, I want to view all projects but only edit comments (not grades) so that I can help students without overriding the lecturer's grading authority.	H
US-TA-02	As a teaching assistant, I want to get notified when a student replies to my comment so that I can follow up quickly.	M

3. Use Cases

Use Case 1: Submit Project for Review

Element	Description
Use Case Name	Submit Project for Lecturer Review
Actor	Student (Project Manager)
Precondition	All required sections (Planning, Requirements, Design, Code, Testing) are marked complete (≥80% filled).
Main Flow	1. Student clicks "Submit for Review" button. 2. System checks completeness. 3. System notifies lecturer via email and dashboard. 4. Lecturer opens project and adds comments/grades. 5. Student receives notification of feedback.
Alternative Flow	If incomplete (<80%), system shows missing sections and prevents submission.

Use Case 2: Generate Final Report

Element	Description
Use Case Name	Generate Final Project Report
Actor	Student (Project Manager)
Precondition	Project is fully completed and graded.
Main Flow	1. Student clicks "Export to PDF/PPT".	2. System compiles all sections (Planning, SRS, Diagrams, Code, Tests, Feedback).	3. System formats according to university template. |4. Student downloads the file.
Postcondition	A professional final report is saved locally for submission.

4. Functional Requirements

ID	Requirement	Source User Story
FR-01	System shall allow lecturers to view all student projects in a dashboard.	US-LEC-01
FR-02	System shall support inline comments on any section.	US-LEC-02
FR-03	System shall calculate final grade based on section grades.	US-LEC-03
FR-04	System shall allow project managers to create Gantt chart timelines.	US-PM-01
FR-05	System shall allow assignment of roles and tasks to team members.	US-PM-02
FR-06	System shall support KPI definition and tracking.	US-PM-03
FR-07	System shall allow developers to link a GitHub repository.	US-DEV-01
FR-08	System shall provide diagram creation/upload for UML, ERD, DFD.	US-DEV-02
FR-09	System shall support test case creation and bug logging.	US-TST-01, US-TST-02

5. Non-Functional Requirements

ID	Requirement	Category
NFR-01	System shall load any dashboard page within 3 seconds on broadband.	Performance
NFR-02	System shall support at least 50 concurrent users (students + lecturers).	Scalability
NFR-03	User passwords and GitHub tokens shall be encrypted (AES-256).	Security
NFR-04	System shall auto-save form data every 30 seconds to prevent loss.	Usability
NFR-05	All diagrams and text shall be exportable to PDF without formatting errors.	Reliability

4. System Analysis – Problem, Use Cases, Requirements, Architecture :

1.Problem Statement & Objectives :

A.Problem Statement

As for IOT systems in industrial fields , there are a huge amount of continuously generated data from multiple devices or machines.

As that most traditional systems :

- Do not support real-time anomaly detection
- Detect failures after they occur
- Lack instant alerting mechanisms

Which may lead to :

- Safety risks
- Equipment damage
- Increased operational costs

There for comes the need for our project which is a real-time anomaly detection system that can recive data streams prcesses them and detects anomalies within real-time with an instant alerting system.

I.Objectives

1. Detect anomalies in real-time
2. Supports streaming data processing
3. Reduce downtime
4. Provide automatic alerts
5. Visualize sensor behavior via dashboards
6. Build scalable ML pipeline

A. Use Case Diagram & Descriptions :

II. Actors:

1. Virtual IoT Sensors

Temperature
Humidity
Vibration
Gas
Smoke

2. System (Automated Platform)

Processing
ML Models
Alerts
Automation
Dashboards

3. System Administrator

Defines thresholds, monitors system

4. Data Analyst

Reviews dashboards, analyzes trends and investigates anomaly patterns

III. Use Cases :

1. Send Sensor Data
2. Process & Analyze Data
3. Detect Anomaly
4. Trigger Alert
5. Store Data
6. View Dashboard
7. Configure Thresholds

IV. Use Case Description

a. **Use Case:** Detect Anomaly

- **Actor**: System
- **Description**:
  The system processes incoming sensor data in real time, analyzes them using the selected model or rule-based logic and classifies it as Normal, Warning, or Critical based on predefined thresholds or model predictions.

b. **Use Case:** Trigger Alert

- **Actor:** System
- **Description**:
  When abnormal values are detected, the system sends an alert message to the alert module and highlights the event in the dashboard.

A.Functional & Non-Functional Requirements :

I. Functional Requirements

- Ingest real-time data from 5 IoT sensors
- Process streaming data using Kafka pipeline
- Process incoming data in real time.
- Detect abnormal readings using:

  1. threshold-based logic
  2. Isolation Forest
  3. Autoencoder
  4. LSTM Autoencoder

- Classify data into:

  a. Normal
  b. Warning
  c. Critical

- Send real-time alerts when abnormal behavior is detected.
- Store processed data in time-series database
- Provide visualization dashboards (Grafana / EDA tools)

V.Non-Functional Requirements

1.Scalability:

Handle thousands of messages per second (via Kafka)

2. Availability:
Operate continuously (a 24/7 system)

3. Low Latency:
Real-time response within seconds

4. Reliability:
No data loss during streaming

5.Modularity:
Components should be loosely coupled (microservices style)

C. Software Architecture

Layer	Component	Github File
Data Generation	Sensor Simulator	sensor_simulator.py
Message Broker	Kafka	(Docker)
Stream Processing	Processor	stream_processor.py
Storage	InfluxDB	influx_writer.py
Alerts	Alert System	alert_system.py
Analytics	EDA Scripts	eda_analysis.py
Visualization	Dashboard	Grafana
Automation	Control Scripts	run_system.ps1

As for our project automation plays a key role in ensuring smooth, continuous operation of system components; Where the system is designed using:

- **Event-Driven Architecture (EDA):**
  Each sensor reading is treated as an event that flows through the system asynchronously.
- **Microservices-Based Design:**
  Each functional component (ingestion, processing, model inference, storage, alerting, and visualization) operates independently and communicates through message streams or APIs.

I. Data Flow Overview

1. **Data Generation**
2. IoT sensors (or simulator) generate time-series data continuously.
3. **Data Ingestion**
   Data is transmitted to a message broker:
   - Current: Kafka
   - Target: Azure Event Hubs
4. **Preprocessing**
   Data undergoes:
   - cleaning
   - handling missing values
   - smoothing
   - feature engineering

5. **Feature Engineering**
   Features such as:
   - lag values
   - rolling averages
     are computed to improve model performance.
6. **Anomaly Detection**
   - Data is passed to anomaly detection models:
   - Isolation Forest
   - Autoencoder
   - LSTM Autoencoder

   Output:

   - anomaly score
   - classification (Normal / Warning / Critical)
7. **Decision & Alerting**
   If anomaly detected:
   - trigger alert
   - classify severity
   - notify user
8. **Data Storage**
   Processed data is stored for:
   - historical analysis
   - dashboard visualization
9. **Visualization**
   Dashboards display:
   - real-time sensor values
   - anomaly rates
   - trends
10. **Monitoring & Feedback Loop**
    System monitors:
    - model performance
    - data drift

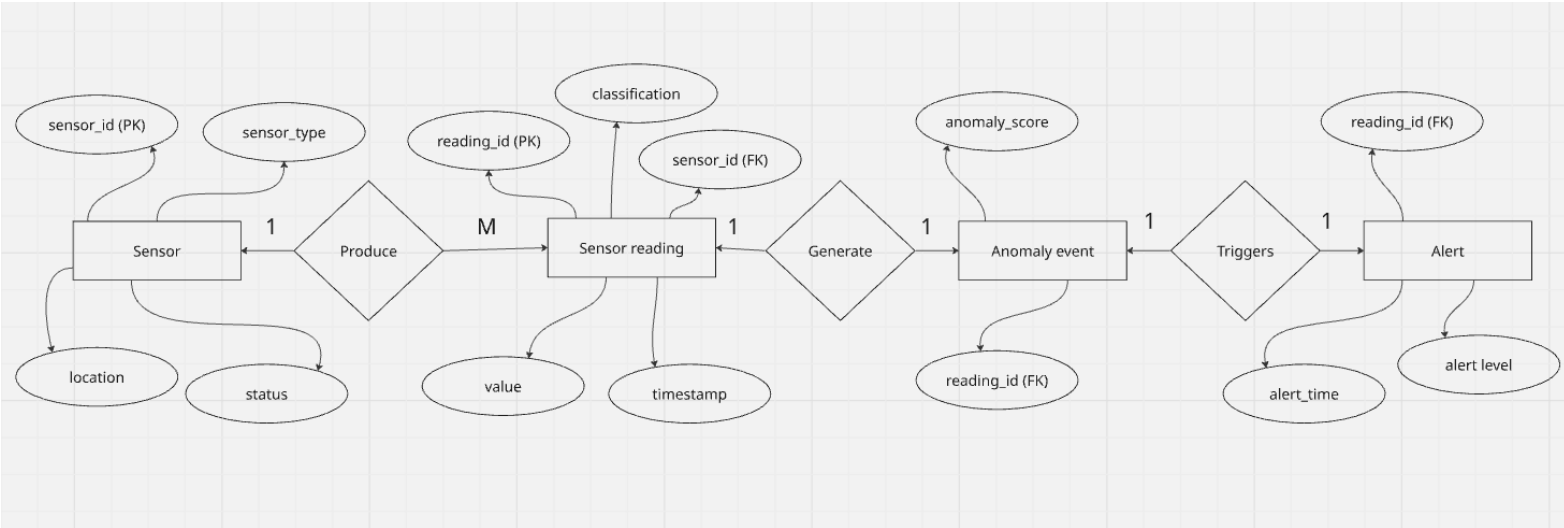
    If drift detected:

    - retraining pipeline is triggered
11. **Deployment & Integration**
    The system is deployed using:

- Docker containers (current)
- Cloud infrastructure (target)

2.Database Design & Data Modeling :

A. ER Diagram:



Some attributes in the ERD (e.g., location) are included as part of a conceptual scalable design.

B. Logical & Physical Schema :

I. Taking sensor data as a measurement;

Field	Type
value	float
anomaly_score	float
classification	string

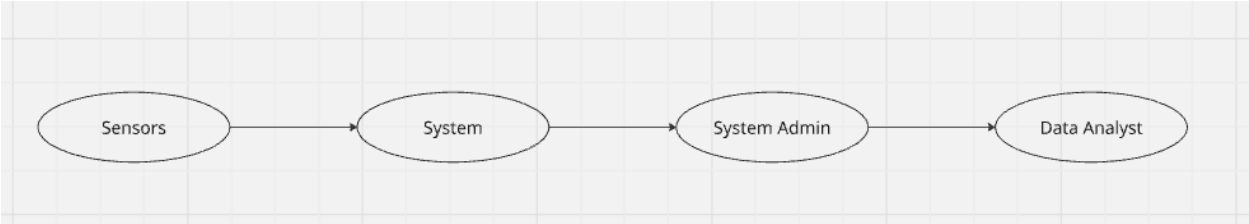
II.Example Record

measurement: sensor_data
tags: sensor_id=1, type=temperature
fields: value=85.3, classification=critical
time: 2026-04-27T12:00:00Z

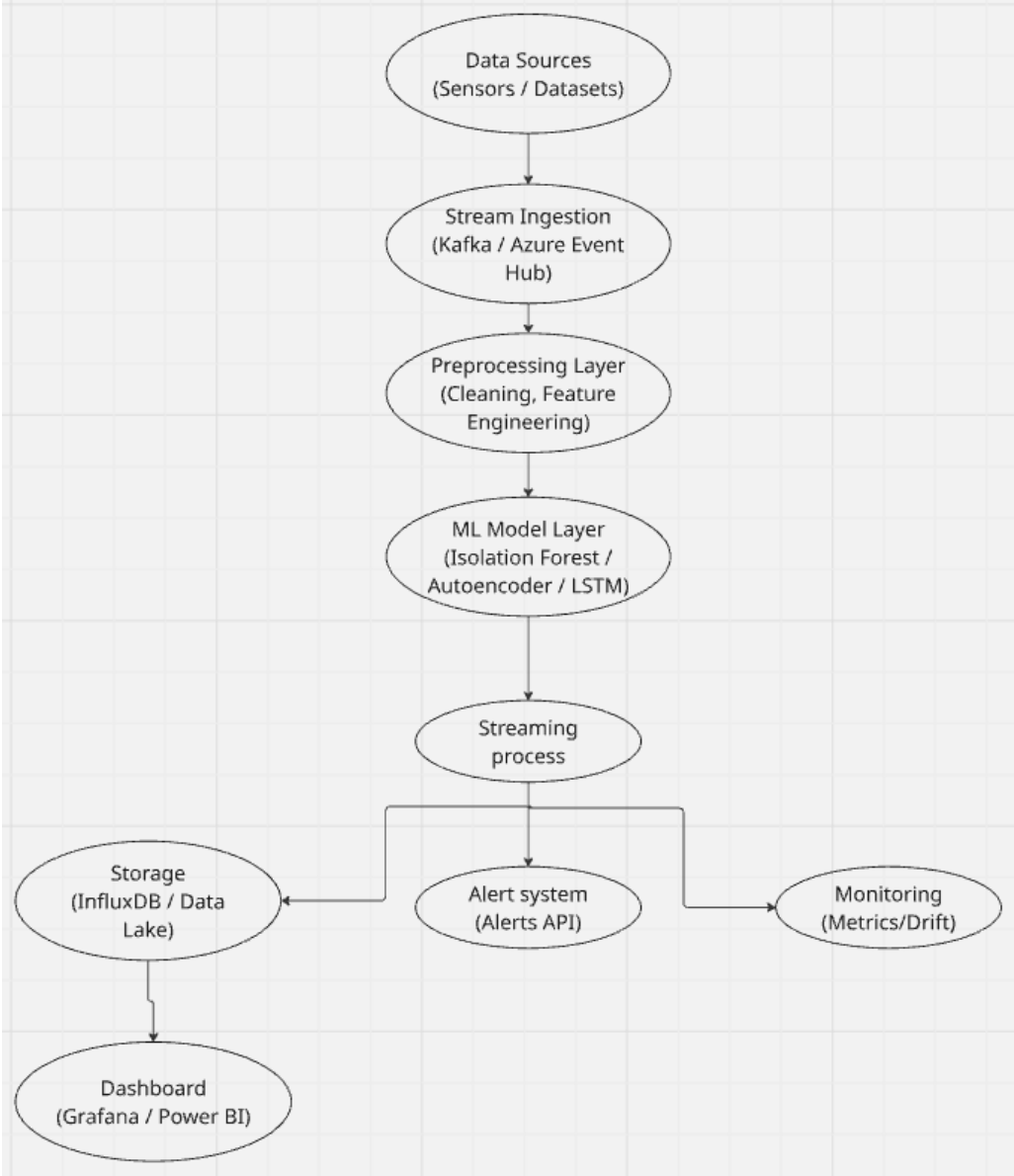
C. Data Flow & System Behavior :

a. DFD (Data Flow Diagram)

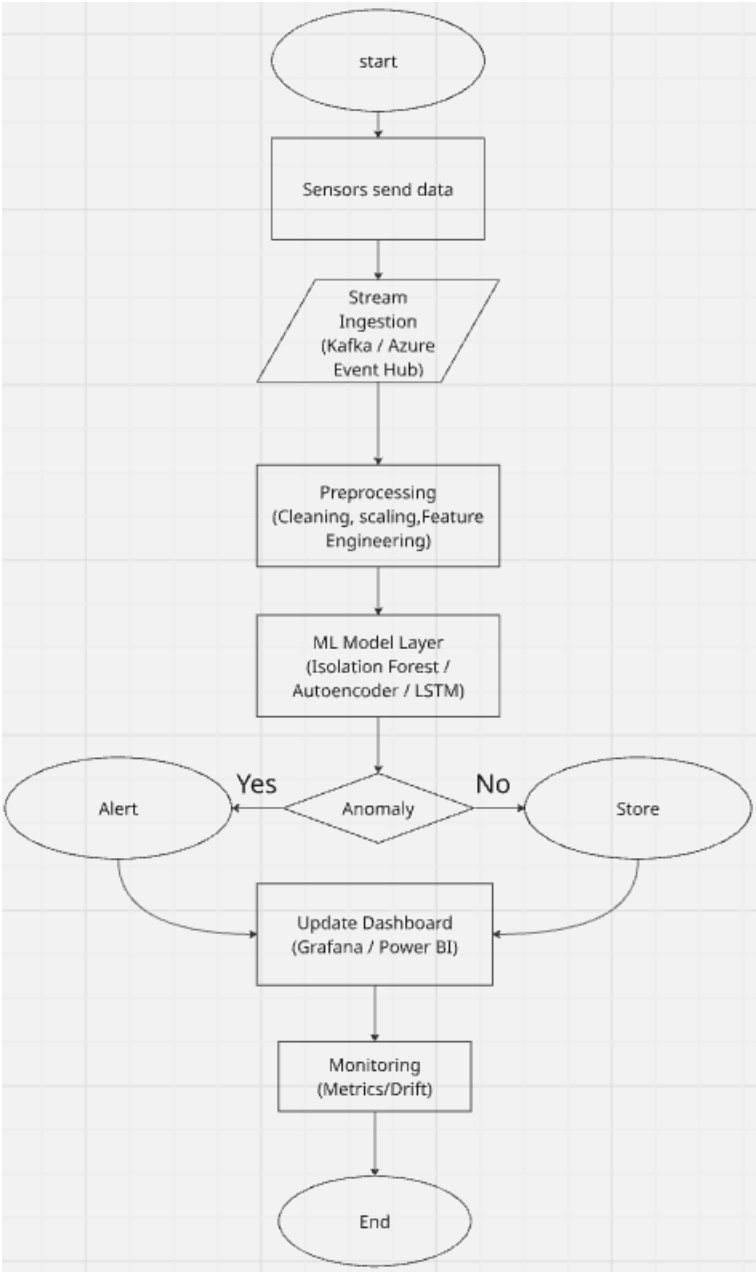
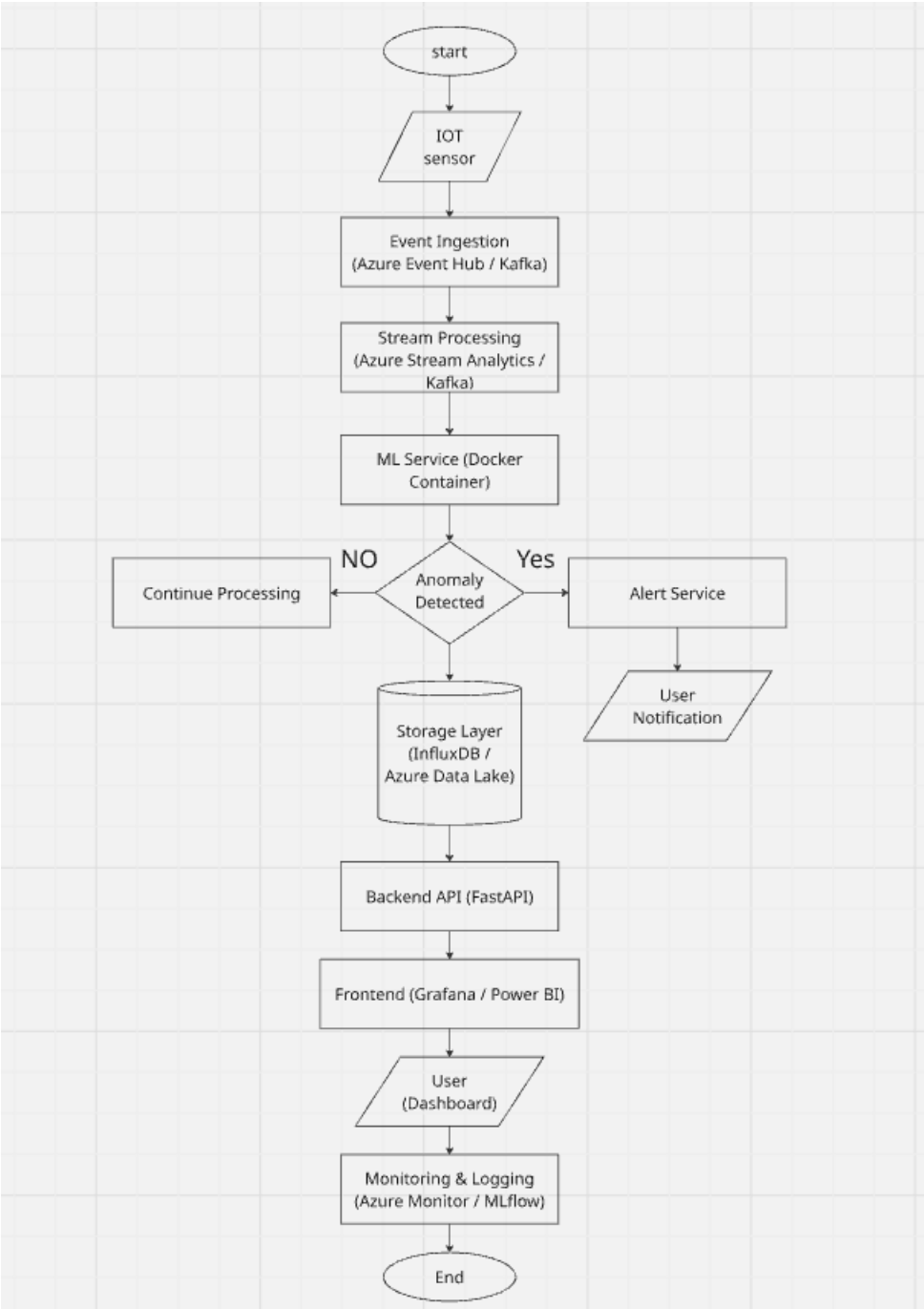
I.Context :



II.Actual pipeline :



b. Sequence and activity Diagram :



D. System Deployment & Integration